Part 0. Triplet overview and integration

Overview

Triplet 04 consists of three services that cooperate through a shared interface. The Twin (team Trinity) simulates a city of 38 nodes, each carrying sensors and, in some cases, an actuator. The City Brain (team CIVIS) reads the sensor data, detects incidents and selects a response. The Guardian (team 9antra) assesses whether each reading can be trusted and signs a permission for the actions it approves. The Twin executes an action only when it receives a valid permission for that exact action.

Architecture

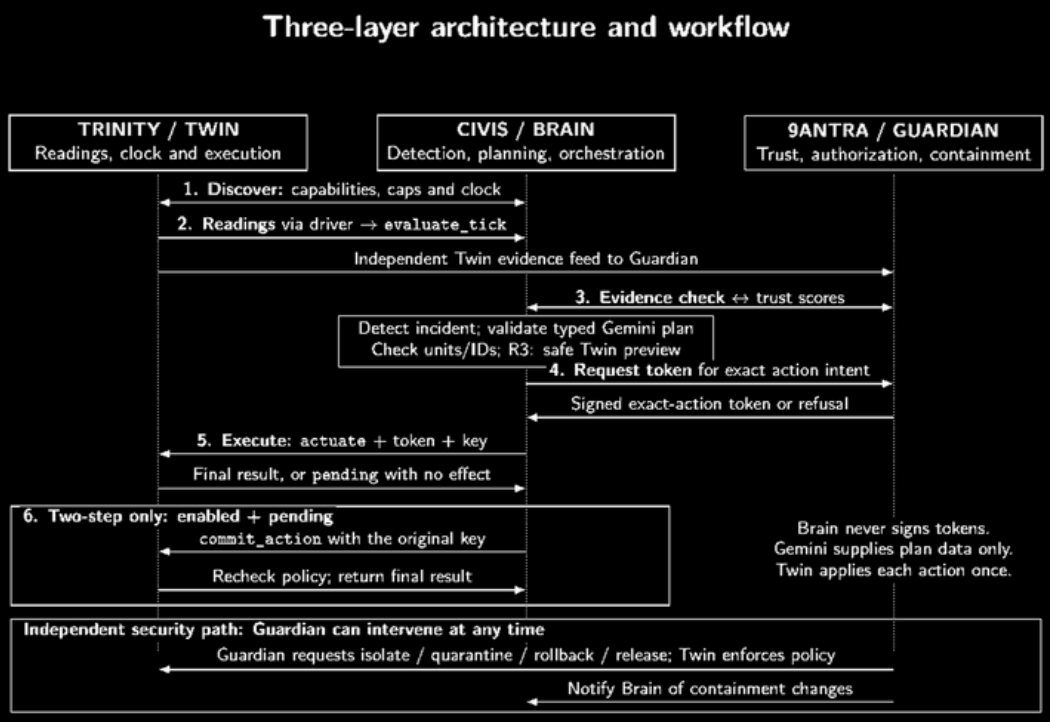


Figure 1. Global architecture. Numbers show the order of a typical action. The dashed arrow is the containment path.

The layers call each other directly. No layer acts as a central relay. The links are:

- **Twin to Brain:** sensor readings, as a live feed (one event per tick) and as one-off tool calls.
- **Twin to Guardian:** the Guardian takes readings on an authenticated HTTP route. In our joint run a Twin-team script forwarded the live feed to that route (see Part 5). The Guardian also calls the Twin's tools for discovery and checks
- **Brain to Guardian:** a request for a trust verdict on the readings behind an incident, followed by a request for approval of one specific action.

- **Guardian to Brain:** the verdict, and either a signed token or a refusal.
- **Brain to Twin:** the action, sent with the token.
- **Guardian to Twin:** containment calls (isolate a sensor, hold a device, correct a reading). The Twin side of these calls is implemented and restricted to the Guardian's key.

The Guardian offers three tools to the Brain: describe_capabilities, get_trust_verdict and authorize_action. The connectivity check of 29 Oct did not exercise the Brain-Guardian link.
If the Guardian does not answer, the Brain holds no token and therefore cannot act, because the Twin refuses every action without one. Scenario T4 checks this case: nothing is committed and the case goes to a person.

How the layers communicate

All three layers are Python services built on FastAPI. Each offers its functions as MCP tools. MCP (Model Context Protocol) is a standard that lets a program publish tools which other programs can list and call. The three layers use the same protocol version (2026-07-28).
Every call carries a key in the Authorization header, with one key per layer, so the Twin can identify the caller. The permission table below shows what each caller may do on the Twin.

Function	**City Brain**	**Guardian**
Read sensors, clock and city map	yes	yes
`preview_action (dry run)`	yes	no
actuate (change an actuator)	yes, with a token	no
Cut off, hold or correct a sensor	no	yes
Read what is cut off	yes	yes

A third key belongs to the scenario tool described in Part 1. The other layers cannot see that tool.
Sensor data reaches the other layers in two ways. The live feed (/events) is an open connection on which the Twin pushes one readings event per tick. Tool calls such as get_readings answer individual requests. Each layer also writes a log file in a shared format, one JSON object per line. These files are merged into one log per run (see Part 1).

One scenario from start to finish: a real traffic jam

The scenario is T2. In the run described here, the Brain and the Guardian are the scripted stand-ins written by the Twin team, so the table shows how the chain looks from the Twin. It does not test the partners' code.

Step	Event	Where it appears
1	The scenario key calls run_scenario with real_jam at node SH-05.	Twin log: scenario event
2	The true traffic values at SH-05 move towards a jam over 10 ticks: congestion index 90, 120 vehicles per minute, 8 km/h. The four linked traffic nodes follow at 35 %.	Live feed and log
3	The Brain observes a congestion index of 80 or more on two consecutive ticks, classifies it as a jam and asks the Guardian for permission.	none yet
4	The stand-in Guardian (set to approve) signs a permission for exactly one action, set_signal_plan with favor_main_road, valid for 60 simulated seconds.	Guardian verdict event
5	The Brain records its decision and calls actuate with the token and an idempotency key. The Twin validates both, commits the change and marks the token as used.	Brain decision event, Twin action event
6	The script merges the part files into logs/T2-<n>.jsonl and checks that exactly one action was committed.	Log file, dashboard timeline

The dashboard shows the chain as arrows from reading to verdict, decision and action. The same run with a fake jam (T3, where a sensor reports false traffic values) ends at step 4. The stand-in Guardian, set to refuse, refuses, the Brain sends no action and refers the case to a person.

Shared decisions

Decision	Agreed approach
Language and protocol	Python and FastAPI for all three layers, MCP protocol 2026-07-28, one agreed list of pinned library versions
Location of the trust gate	In the Twin, which refuses any action without a valid token
Tokens	Signed by the Guardian (Ed25519), valid for one exact action, at most 60 simulated seconds, usable once
Caller identity	One secret key per layer
Actuator call	`A single actuate call with action, targets, token and idempotency_key`
Repeated requests	The same idempotency key sent twice returns the first answer
Risk levels	Signal plan R1, dispatch R2, valve R3 (defined by CIVIS)
Limits per request	Action cap of 30 % of a domain's nodes, isolation cap of 50 % (Power may isolate all of its sensors)
Containment	Callable only by the Guardian, counted against its own caps
Dispatch target	The place field is named destination
Logs	JSON Lines, one part file per layer, merged into one file per run
Data format	UTF-8 JSON, snake_case names, simulated UTC time plus an integer tick, a missing value is null
Unknown fields	Ignored on input. Missing or invalid required fields are rejected with an error code.

Part 1. The City (Twin)

Role and goals

The Twin is the simulated city. It generates the sensor readings on which the Brain and the Guardian operate, and it is the only layer in which an actuator command changes state. It does not decide which action to take and it does not judge the trustworthiness of a reading. Those tasks belong to the other two layers.

The design pursues four goals:

1. Provide readings with realistic imperfections (bias and noise), so that the other layers work on data that is not ideal.
2. Produce faults and incidents on demand and reproduce them exactly: a real traffic jam, or a sensor that reports false values.
3. Reject every action that does not carry a valid, signed permission from the Guardian.
4. Leave verifiable evidence. A scenario starts with one command and ends in a log file in the repository, and a dashboard shows the chain from reading to action.

Architecture

City plan and zones

The city is synthetic and generated from a seed (42 in all runs), so every machine builds the same city. It has 38 nodes: Traffic 11, Water 9, Power 9, Air Quality 8 and Emergency 7. Six nodes belong to two domains at once, for example a pumping station that depends on power, so the domain counts add up to 44. The nodes are distributed over 6 zones and connected by 10 roads and 67 links. A link joins two neighbouring nodes, so a layer can query what lies next to a node. A sixth layer, Land, has no nodes and serves as the base map.
Each node has a fixed GPS position (anchor 36.0 N, 10.0 E, one map unit equals 5 m, which makes the city about 9.6 km by 5.4 km). The city, its streets and its coordinates are invented and do not represent real data.

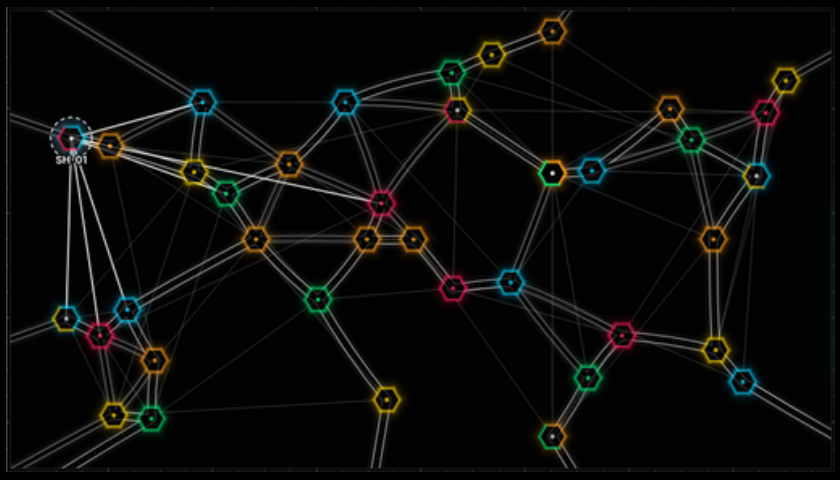


Figure2 . Dashboard city map. Each hexagon is a node, coloured by domain. Lines are roads and links. SH-01 is selected.

Sensors and actuators

Domain	Sensors (unit)	Actuator	Risk level	Dry run first
Traffic	vehicle_count (vehicles/min), avg_speed (km/h), congestion_index (percent)	set_signal_plan: normal, favor_main_road, all_red	R1	no
Water	water_level (cm), flow_rate (L/s)	set_valve_position: 0 to 100 %	R3	yes
Power	load_kw (kW), voltage_v (V)	set_grid_switch: closed, open	none	yes
Air Quality	pm25 (ug/m3), co_ppm (ppm), temperature_c (C)	none	n/a	n/a
Emergency	emergency_calls (calls/min by type: accident, fire, flood, medical), units_free (police, ambulance, fire)	dispatch (unit type, destination, units)	R2	no

The Twin produces 120 readings per tick in total. The City Brain team defined the risk levels. A dry run (preview_action) reports what a command would change without changing it.
Two kinds of limit restrict the effect of a single request. An action may touch at most 30 % of a domain's nodes, with a minimum of 2: Traffic 3, Water 2, Power 2 and Emergency 2. Isolating sensors is capped at 50 %: Traffic 5, Water 4, Air Quality 4 and Emergency 3. Power is the only exception and may isolate all 9 nodes after a serious electrical threat, because isolating a sensor removes its data and never opens a switch.

Simulation model

A reported sensor value is the sum of three components: the true value, the bias of the individual device and random noise. The true value follows slow waves of 5 to 20 simulated minutes around a base level. Only the reported value leaves the Twin. The true value stays inside the Twin and is written to a private file that is never merged or shared. A test verifies that no tool, feed or log line exposes it.

A reported value depends only on the seed, the device and the tick number. Two runs with the same seed and the same scenarios therefore produce identical data, which makes scenarios repeatable and logs comparable. The clock is simulated. One tick equals one simulated second and runs at real speed, starting at 2026-10-05T08:00:00.000Z. Every reading carries the tick number and an RFC 3339 timestamp. congestion_index is a third traffic sensor, in percent, that follows the vehicle count at the same node.

How scenarios are played

Scenarios are started through a single tool, run_scenario. Only the scenario key can see it. For every other caller it fails with the same error as a tool that does not exist.

Scenario	Effect
`fake_reading`	A sensor reports a chosen false value while the true value stays normal (default: water level of 180 cm at WAT-01).
`stuck_sensor`	A sensor repeats its last value and stops changing.
`replay_exact`	An earlier reading is sent again with its original id and time.
`real_jam`	A real traffic jam. The true values at one node (default SH-05) move towards a congestion index of 90, 120 vehicles/min and 8 km/h. The four linked traffic nodes follow at 35 %. The jam ramps up over 10 ticks, holds for 40, fades over 10 and then ends on its own.
`list, stop, reset`	List active scenarios, end a scenario, or start a clean run with a new run id and tick 0.

A fault always starts at least one tick in the future, so readings that were already sent never change. The jam is the only scenario that changes true values. It ramps up gradually because an abrupt step would look artificial to a rate-of-change check.
Each scenario in the test list has its own script in scenarios/, T1 to T7. A script checks that the Twin is running, resets the run, triggers the scenario, saves the merged logs as logs/<ID>-<n>.jsonl, checks the expected result and prints PASS or FAIL.

The seven test scenarios and their expected results are in Part 4.

Components and dashboard

The code is divided so that the simulation, the partner interface and the user interface remain separate:

- **Simulation:** city generator, sensor model, clock and scenario book. These components have no knowledge of the partners.
- **Action gate:** validates every actuate call and is the only code that changes actuator state.
- **Containment:** keeps the list of isolated and held devices. Only the Guardian's key can change it.
- **Interface:** the MCP server at /mcp, the live feed at /events (Server-Sent Events, an open HTTP connection on which the server pushes events) and /health.
- **Run log:** writes the Twin's events, one JSON object per line.
- **Dashboard:** a separate process that reads the log files and the live feed. It never calls a tool.

The dashboard shows a timeline with four lanes (Twin readings, Guardian verdicts, Brain decisions and Twin actions) and arrows from cause to effect. Selecting an event highlights its whole chain. Beside it is a map of the 38 nodes. Each node shows its trust state, and a layer switch fades whole domains in and out. A status chip in the top bar shows whether the live feed is running. The dashboard needs no internet connection.

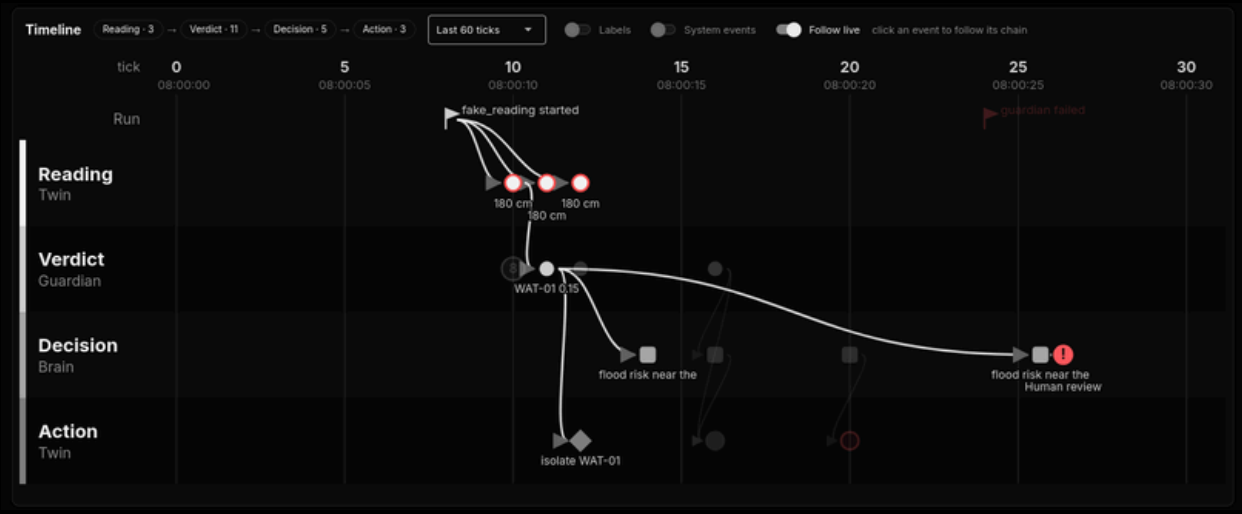


Figure 3. Dashboard timeline during a fake_reading scenario at WAT-01 with the stand-in Brain and Guardian. The false 180 cm readings (Reading lane) lead to a Guardian score of 0.15 (Verdict lane), the isolation of WAT-01 (Action lane) and a Brain decision that ends in human review (Decision lane). The 0.15 score is the stand-in's fixed refusal score, not a real judgement.

Tech stack and why

Component	Choice	Reason
Language	Python 3.12	Agreed by all three teams; fast to develop and test
Tool interface	mcp 2.3.0, protocol 2026-07-28	One version for all layers; tools describe themselves
Web server	FastAPI 0.142.2, Uvicorn 0.54.0, sse-starlette 3.5.0	Serves tools, live feed and health check from one application
Input validation	Pydantic 2.13.5	Every input is validated and invalid input receives a clear error
Configuration	PyYAML 6.0.3, pydantic-settings 2.15.0	Addresses and keys are settings, not code
Signatures	`cryptography 50.0.2`	Verifies the Guardian's Ed25519 signatures
Dashboard	NiceGUI 3.18.0	Pure Python and works offline
Tests	pytest 9.1.1, pytest-asyncio 1.4.0	544 tests pass and 18 are skipped (8 need a PostgreSQL database, 10 need a Redis server)

All versions are pinned, so a fresh clone installs the tested versions. Two storage options exist as extras and are disabled by default: a history database (TimescaleDB on PostgreSQL) and Redis for used tokens and repeated requests.

Interfaces in/out

The Twin sends readings (the live feed and get_readings), the city map (list_nodes, get_city_graph, get_nearest_nodes), the clock, the list of actions with their limits, the containment state and its own run log. get_city_graph and get_nearest_nodes are an addition agreed with the City Brain team in a weekly meeting.

The Twin receives tool calls. The Brain sends actuate and preview_action. The Guardian sends isolate_sensor, quarantine_device, release_device and rollback_reading. Only the scenario key sends run_scenario. A call without a key returns UNAUTHENTICATED and a call from the wrong layer returns FORBIDDEN. The Interface Card (twin/INTERFACE.md) gives one example of each message and explains how to point the Twin at another partner.

Security measures

- **One key per caller.** Keys are compared in constant time and kept out of the repository. The Twin refuses to start if a key is missing, too short or identical to another key.
- **A gate in front of every action.** actuate runs its checks in a fixed order: the request is well formed, the idempotency key, the token, the dry run where one is required, and the cap. Without a valid token no action is executed.
- **Tokens.** The Twin has two modes. Unsigned mode accepts the team's own test token. Signed mode verifies the Guardian's g1 token (Ed25519). In both modes the Twin rejects a token that has expired, has already been used, belongs to another run, names a different action or key, has a lifetime over 60 seconds, or carries a score outside 0 to 1 (including NaN). A token is only marked as used when the action commits.
- **Exactly-once execution.** A repeated request with the same idempotency key returns the first answer. The same key with a changed request is rejected.
- **Fail-safe behaviour.** With Redis enabled, actuate refuses and changes nothing when Redis cannot be reached (STATE_UNAVAILABLE).
- **Containment restricted to the Guardian.** An actuate aimed at a node with a held device is refused (DEVICE_QUARANTINED) and the command is kept for the Guardian to read.
- **True values stay inside.** True values are written only to a private file that is never merged.
- **Hidden scenario tool.** No other layer can start an attack.
- **Input and output handling.** Unknown fields are ignored, missing or invalid ones are rejected with a code, and all log text is escaped before the dashboard displays it. This was tested with script and image payloads.

On 29 Oct, in a check with the partners' packages, a real Guardian token was accepted by the Twin in signed mode. A second use of the same token was refused (TOKEN_REUSED) and a token signed with a different key was refused (TOKEN_INVALID).

Known limitations

- An accepted action is stored and logged, but nothing in the city reacts to it. A signal change does not ease the jam.
- The traffic jam is the only real incident. Floods and power surges exist only as fake readings.
- The sensors follow slow waves and not a full daily cycle, because a day would not be visible in a demonstration at real speed.
- The default configuration accepts the unsigned test token so that the Twin can run on its own. Signed mode (tokens.mode: signed and the Guardian's public key) is a configuration setting.

- A restarted Twin keeps its run id and, with the default in-memory state, loses the record of used tokens and dry runs. A reset starts a clean run. Redis keeps the tokens across a restart.
- Containment requires only the Guardian's key and no token. The caps limit the damage if that key is exposed.
- Reconnecting to the live feed resumes at the current tick. Missed ticks are not replayed, so a layer should call get_readings after reconnecting.
- The Twin logs the readings that a scenario touched, not every normal reading. The dashboard shows only what the layers logged.
- The automatic log merge only finds partner log files in the Twin's own folder. Files from another folder must be copied or passed with --parts.
- The dashboard's trust bands are display settings in the Twin's configuration and are not read from the Guardian.
- The stand-ins for the Brain and the Guardian run the Guardian inside the Brain script, so they do not test the network call between the two.
- The city, its streets and its GPS positions are invented.

Part 2. The City Brain

Role and goals

CIVIS is the City Brain. It receives Twin readings and simulated-clock context, detects an incident, validates a typed response, asks 9antra Guardian for trust and exact action authorization, then asks Trinity Twin to execute the approved command. CIVIS does not sign approval tokens, execute containment, or bypass Twin limits.

The primary mock covers five scenarios:

ID	Domain	CIVIS response
S01	Traffic congestion	R1 set_signal_plan request after persistent congestion
S02	High water	R3 virtual set_valve_position request with a required Twin preview
S03	Power anomaly	Evidence warning only, with no grid action
S04	Air quality	Sustained PM2.5 alert, with no actuator
S05	Medical incident	R2 dispatch request for one ambulance to the reported destination

S01 to S05 are the Brain's five domain scenarios. T1 to T7 in Part 4 are the test scenarios.

The goal is safe orchestration. An incomplete reading, stale evidence, an unavailable peer, a rejected trust result, an expired preview, or a changed incident blocks the action and records the reason.

Architecture

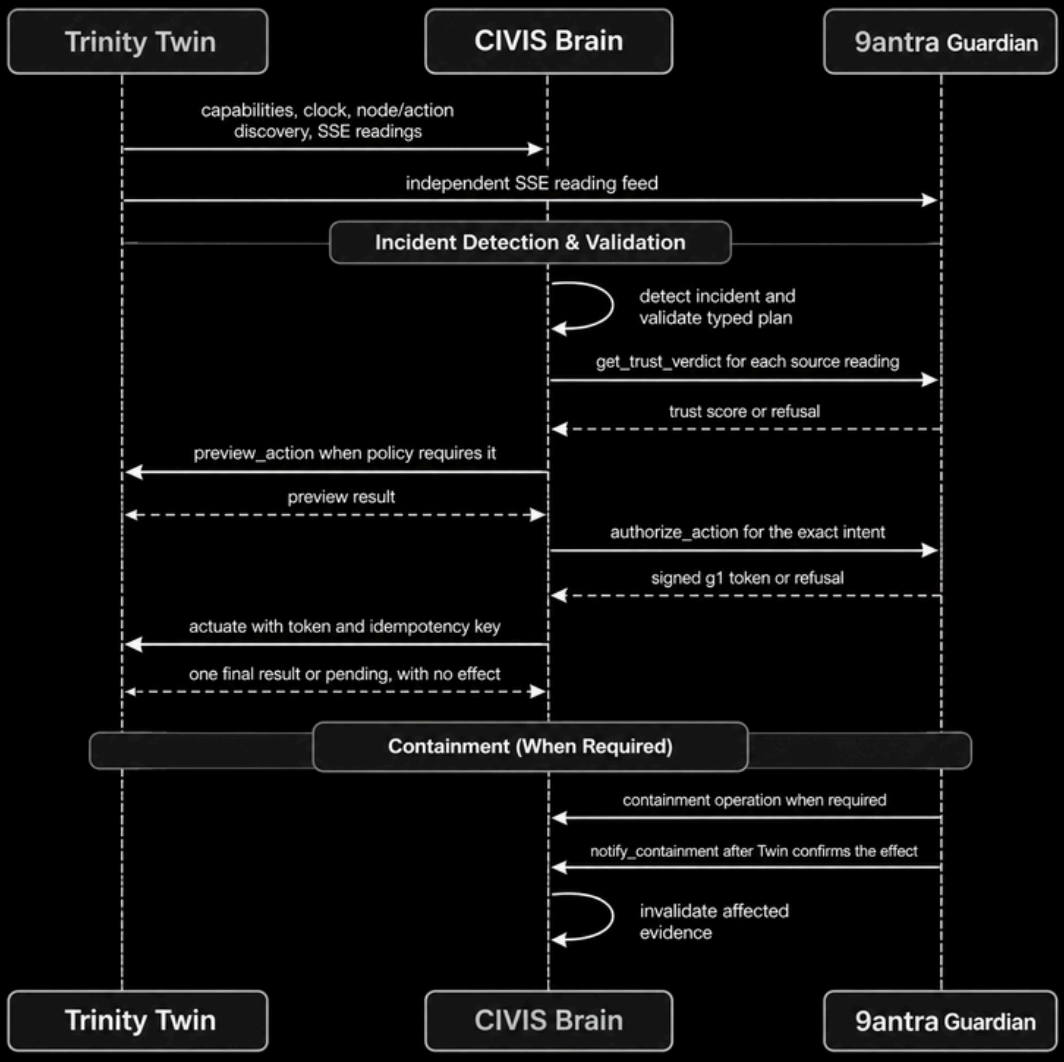


Figure 4. Message flow of the City Brain with the Twin and the Guardian.

The runtime separates transport, scenario detection, planning, validation, Guardian authorization, and Twin execution. Twin discovery supplies the current nodes, actions, caps and clock. CIVIS retains original reading IDs, units, null values and multi-domain membership throughout the flow. It uses Twin simulated time for evidence freshness and token lifetime, while network calls and Gemini timeouts use real monotonic seconds.

Gemini produces a constrained typed plan. CIVIS validates its action, targets, units, risk tier, evidence IDs, caps and preview before contacting Guardian. Guardian approval is tied to the exact action intent. CIVIS forwards the original signed g1 token to Twin unchanged. Twin remains the final execution gate.

Tech stack and why

Component	Choice	Reason
Service and validation	Python 3.12, FastAPI 0.142.2, Pydantic 2.13.5	Typed service boundaries and explicit schema validation
Partner tools	MCP 2.3.0, Streamable HTTP, SSE	The agreed tool protocol and live Twin reading feed
Network client	HTTPX 0.28.1	Bounded authenticated calls to Twin and Guardian
Planning	Google Gen AI 2.28.0 with Gemini Flash-Lite	Constrained typed planning, never authorization
Quality checks	pytest 9.1.1, pytest-asyncio 1.4.0, Ruff 0.16.10	Repeatable regression checks and linting
Optional graph study	Small NumPy-trained GraphSAGE-style traffic forecaster	Read-only geographic context experiment without changing the service dependency lock

The optional traffic forecast consumes five complete Twin ticks of vehicle_count and avg_speed for the discovered Traffic nodes. Twin neighbour links are geographic proxies, not confirmed road links. The forecast and its separate Gemini interpretation are shadow-only: they do not create incidents, assign risk, authorize commands, or call an actuator.

Interfaces in/out

Direction	Interface	Purpose
Twin to CIVIS	SSE reading feed and Twin MCP tools	Readings, clock, nodes, actions, caps, previews and final execution results
CIVIS to Guardian	`describe_capabilities`, `get_trust_verdict`, `authorize_action`	Discover policy, obtain a verdict for each original reading ID, request exact action authorization

Guardian to CIVIS	`notify_containment`	Notify an already confirmed Twin containment result so CIVIS invalidates affected evidence
Caller to CIVIS	`get_capabilities`, `evaluate_tick`, `get_active_incidents`, `explain_decision`, `notify_containment`	Brain public tool inventory, tick evaluation, decision inspection and Guardian-only containment notice

Partner addresses and Bearer keys are local configuration, never hard-coded. The standard peer call limit is three real seconds. Gemini planning has a 60-real-second limit. The default evidence age is 10 simulated seconds, extended to 60 only by the explicit mock profile. A Guardian token is valid for at most 60 simulated seconds. A lost execution reply keeps the outcome uncertain, so CIVIS reuses the original idempotency key and does not invent a new command.

Security measures

- Typed input models reject malformed data before it reaches planning or execution.
- CIVIS validates fresh evidence, current containment state, target membership, action caps and required previews before authorization.
- Guardian authorizes an exact action, risk tier, domain, supporting reading IDs and idempotency key. CIVIS never receives a Guardian signing key.
- Twin enforces the signed token, simulated expiry, one-time use and idempotency at execution time.
- Any missing evidence, score, required preview, valid token, peer response or current clock blocks the action.
- Guardian-only containment notices are deduplicated by event ID. A valid notice invalidates the affected CIVIS evidence; CIVIS never calls Twin containment tools itself.
- .env files, Bearer credentials, signing material and full action tokens are excluded from Git and from shared logs.

Known limitations

The Brain primary mock implements only S01 to S05. S06 to S08 live policies remain disabled. S02 proves a pre-approved virtual command for WAT-01 at 40 percent in the mock seed; it is not a hydraulic safety model. S03 and S04 are alert paths without actuation. Runtime state is in memory, so a restart requires reconciliation or a new Twin run.
The traffic graph model was trained on deterministic Twin telemetry.

It has no demonstrated improvement over its non-graph baseline and makes no real-city prediction claim. It is disabled by default.
As of the 3 November Brain candidate check, CIVIS regression tests passed (555) and Ruff passed. The gate against the current tagged Twin and Guardian recorded 22 passing and 8 failing checks. Full three-layer acceptance is therefore not claimed. The remaining failures include Guardian prerequisites for S02 and S05, units_free assessment, rollback freshness, signature-validation policy, and containment propagation. Each must be retested against a newly tagged partner build before it can be closed.

Part 3. The Guardian

Role and goals

The Guardian checks whether sensor evidence is trustworthy and whether a proposed action meets the agreed policy. It receives readings from the Twin, scores them using deterministic validation and reputation rules, and provides the Brain with trust verdicts and action authorization. It does not choose or execute city actions. The Brain plans; the Twin decides whether a command is valid and changes the simulated city.
The Guardian also handles containment. It delegates isolation, quarantine, rollback and release to the Twin. After the Twin confirms an effect, the Guardian mirrors that state for its own scoring and notifies the Brain. Its main goals are to keep evidence traceable to its readings, refuse actions that do not meet policy, and keep the authorization decision separate from the final execution gate in the Twin.

Architecture

Twin readings enter the Guardian through the Bearer-authenticated SSE feed and are normalized into the Guardian's reading model. The Guardian checks reading structure, units, ordering and duplicate delivery before scoring. Conflicting content under an existing reading ID is rejected; skipped tick gaps are recorded.
The scoring service applies deterministic validation, correlation and reputation checks. The Random Forest model may add an advisory, but its output has zero numeric weight in the trust score. Current trust states are Trusted at 0.7 or higher, Degraded above 0.3 and below 0.7, and Untrusted at 0.3 or lower.
For authorization, the Guardian checks the current verdict and its supporting reading IDs, the action's domain and risk, the exact parameters and targets, and the Twin's discovered action and containment state. If the checks pass, the Guardian signs a g1 action token with Ed25519. The Twin verifies the token and remains responsible for enforcing its action limits and executing the command.

The tagged S02 water action is restricted to set_valve_position on WAT-01 at 40 percent, with a Twin preview of the exact command. The Brain makes that preview call; the Twin enforces its 60-tick and one-commit rule. For S05, the Guardian requires an Emergency/R2 ambulance dispatch to the discovered carrier, for one unit, with both the medical incident reading and a trusted ambulance units_free reading included as evidence.
For containment, the Guardian calls the Twin's published tools and waits for confirmation before notifying the Brain. The Brain notice uses a stable event ID and maps the confirmed Twin result to isolated, quarantined, corrected or released.

Tech stack and why

The Guardian is implemented in Python and serves its APIs and MCP tools through FastAPI. Pydantic validates message schemas. The MCP SDK provides the Brain-facing tool interface, while sse-starlette supports streaming integration. Ed25519 signing and verification use cryptography. Scikit-learn, NumPy, pandas and joblib support the optional Random Forest advisory.
The dependencies are pinned so the service can be installed consistently. These tools separate input validation, policy decisions, signed authorization and advisory model output.

Interfaces in/out

Link	**Information exchanged**
Twin → Guardian	Bearer-authenticated reading events, including run and reading IDs, tick, timestamp, sensor, channel, value, unit and location
Twin ↔ Guardian	MCP discovery and checks for actions, clock, nodes, containment state, and current units_free evidence
Brain → Guardian	Capability and advisory requests, trust-verdict requests, and action-authorization requests
Guardian → Brain	Trust verdicts, authorization decisions and signed action tokens; containment notices after a confirmed Twin effect
Guardian → Twin	Signed action tokens are supplied by the Brain to the Twin; the Guardian itself sends containment tool calls to the Twin

For S05, the Guardian scores units_free using its channel and unit: channel=ambulance, unit=units.

It retains the reading ID and checks the carrier's latest Twin reading before authorizing. If the feed is one Twin tick behind, the Guardian accepts the evidence only when the current unit count is unchanged.

The public Brain-facing MCP tools include capability and advisory tools, get_trust_verdict and authorize_action. The test-only token verification tool is disabled by default.

Security measures

Each service link uses its own Bearer credential. The Twin feed is Bearer-authenticated; it does not carry signed readings in twin-mock-3. This is separate from the Guardian's Ed25519 g1 signature on action tokens, which the Twin verifies with the Guardian's configured public key.

Token claims bind the run, action, targets, parameters, idempotency key, supporting reading IDs and key ID. Tokens use Twin-simulated time and are limited to 60 simulated seconds; the Twin enforces one-time use when it accepts a command. If Twin discovery or the required signing key is unavailable, the Guardian fails closed.

Reading-signature requirements are controlled by the Guardian's configuration, not by a caller-supplied per-reading switch. Any signature that is supplied is validated. Demo mode bypasses authentication on protected routes and is intended only for local demonstrations; it is off by default.

The Guardian records reading evaluations, authorization decisions and containment outcomes in JSON Lines audit logs. It does not put full action tokens in those logs.

Known limitations

The Random Forest was trained on synthetic data; it is not evidence of real-world security performance. It has no units_free training feature, so its advisory is unavailable for that sensor. Deterministic Guardian scoring remains available.

Readings, verdicts, reputation and token replay state are held in memory, so a restart does not restore that state. Local checks use in-process peers or mocked partner effects and do not establish that a live Twin and Brain deployment has completed a joint run. The latest Guardian follow-up log records compilation and Brain-notice mapping and configuration checks as passed; it does not record a pytest run or live joint gate.

Part 4. Test Book

Evidence rule

The Test Book covers each layer alone and the interactions between Twin, Guardian and CIVIS.

Each test starts from one trigger and produces a run ID, and every participating layer writes its own JSON Lines evidence file. A result is real PASS only when the logs of the required real layers share that run ID and show the required sequence. A stand-in result proves the local harness only and is never presented as full three-layer acceptance. A run is labelled real, stand-in or not run, which means the required same-run evidence has not been collected.

Scenarios

ID	Name	Layers	Required outcome	Status
T1	Calm city	Twin, Brain	Normal readings create no incident and no Twin action	real Partial
T2	Approved traffic congestion	Twin, Guardian, Brain	Sustained congestion is detected, the Guardian authorizes one signal-control action, and the Twin commits it once	real PASS
T3	Untrusted traffic reading	Twin, Guardian, Brain	The Guardian refuses or marks the reading untrusted, the Brain blocks, and the Twin commits nothing	real PASS
T4	Guardian unavailable	Twin, Guardian, Brain	With the Guardian silent, the Brain blocks and records the failure,and the Twin commits nothing	stand-in PASS
T5	Approval-token replay	Twin, Guardian	The first use of a token can commits once, and reusing it is refused with TOKEN_REUSED	real PASS
T6	Stuck sensor containment	Twin, Guardian, Bran	The Guardian requests containment, the Twin confirms it, and the Brain invalidates the affected evidence	stand-in Partial
T7	Duplicate command	Twin, Guardian, Brain	The same idempotency key gives one Twin effect and the retry on returns the original result	stand-in PASS
T8	Tampered approval token	Twin, Guardian	One valid control request commits once, and the request with a changed signature is refused with TOKEN_INVALID and has no effect	real PASS

Procedure

1. Start the tagged Twin, Guardian and CIVIS services with the agreed private configuration.
2. Record the run ID, then issue the single scenario trigger.
3. Save each layer's log as run-logs/<run_id>.<layer>.jsonl.
4. Merge only the logs of that run ID and compare the ordered events with the scenario's pass rule.
5. Mark the scenario real PASS, stand-in PASS, FAIL or not run. A missing log, a different run ID or a missing required event is not a real pass.

For each scenario, the trigger, the expected behaviour of each layer, the pass rule and the evidence files are defined in TESTBOOK.md in the repository.

Part 5. Collaboration process

How we work

One rule: an agreement does not count until it is written down, has an owner and a date, and sits where all three teams can open it. We talk on Discord and WhatsApp, share files on a shared Google Drive, and meet in a weekly call. The day and time change each week. Each step leaves a document.

Step	Practice	What you can open
Capture	A meeting page filled in live: decisions, owners, deadlines, open questions	The written record of each call
Store	One shared drive for every document that is not code	Decision report, records, sensor table, check reports
Decide	Numbered decisions, cited by number	Decisions D1 to D10
Assign	One owner, one deadline and a reminder per task and per shared file. Each call opens with the task list	Task list in each record
Specify	Technical agreements live in the repo beside the code	Sensor table, INTERFACE.md files, pinned versions
Verify	Acceptance lists and joint checks with the partners' own packages	Acceptance list, check reports, run logs
Fall back	The others switch to a stand-in for an unready layer and mark its events stand_in	Stand-in scripts and logs

All code is in one repository, triplet-04, with one folder per team. A change in the shared root (scenarios/, logs/) needs approval from the other two teams.

Problems and what we did

Problem	What we did
The first joint run failed because nobody had seen the others' code	Integration became the only priority, and ten decisions were written down the same day
The acceptance list showed our Twin accepted five bad tokens and could not read the Guardian's token format	We fixed all five rows in the next build
The Brain and the Guardian used different tool names and fields	We proposed that the Guardian keeps its names and the Brain adapts. Parts 2 and 3 now use the same tool names.
The Guardian had no reader for the Twin's live feed, so the Brain's verdict requests got 'verdict_not_found'	On 7 Nov the Twin team wrote a bridge script that forwards the feed, and reported it to 9antra. With it, verdict requests succeed.
The real jam (T2) was blocked in the Brain with 'A touched domain has no discovered action cap'.	Reported to CIVIS on 7 Nov with the logs. The Twin publishes no action cap for air quality,and the shared node SH-05 belongs to traffic and air quality. So the block is shared between the Twin and CIVIS.

Evidence of a joint integration run

The connectivity check ran the partners' packages unchanged: 28 passed, 0 failed, 1 skipped. It covered the live feed, the tools, the city graph, the Brain's start-up checks and a real Guardian token, which the Twin accepted and then refused on reuse. It did not cover the Brain acting through the Guardian.
On 7 and 8 Nov we ran the real traffic jam (T2) with all three real layers five times (T2-5 to T2-9). Only T2-8 and T2-9 are fully clean, because the first runs picked up old log data. Each link between layers works at network level. The chain stops before the Twin commits an action. The Guardian handles readings more slowly than the Twin produces them, so some verdict lookups fail. The Brain's plan for the shared node SH-05 is refused because there is no air quality cap, and the Brain's AI budget runs out. We have the logs and know who owns each issue. We have not recorded a successful joint run. T1 passed with the real Twin and Brain. T4 and T6 passed only with a stand-in Guardian. The logs are in logs/.